

Rapport de stage de fin d'études (2010-2011)
M2 Ingénierie des réseaux, ISTIC, Université de Rennes 1
Équipe TPN/CORE/MOVIE, Orange Labs Lannion

Responsable de formation : Adlen KSENTINI
Encadrement entreprise : Irène GROSCLAUDE
Suivi universitaire : Yassine HADJADJ-AOUL

Développement et intégration de fonctions multicast dans des routeurs ouverts pour la gestion du trafic vidéo événementiel

par Damien LE QUÉRÉ
<damien.lequere@gmail.com>

Stage effectué du 14/03/2011 au 20/08/2011 à Lannion
Soutenance à Rennes, le 1er septembre 2011

Abstract

This document represents the results of a six-month internship at Orange Labs in Lannion. This course completes the M2 IR (professional master degree in computer science, specialized in network engineering) that I am preparing at *ISTIC* of the *Rennes 1* University. The dual purpose of this report is to describe the context of the course and to relate the work done.

On the substance, a summary of the current context of use of the IP multicast and its main issues will be made to show how it is – technically – possible for an operator to dynamically meet a ponctual multicasting need. One can imagine some external or internal needs of an operator. As a new broadcast service, for example, or a failover process to multicast to conserve network resources. Initiated by the *Envision* project, we will explain how the proposed solution is interesting for this project and how it has been integrated into it.

My work during this internship was to study and develop two new elements that the proposed solution involves in an operator network: the « Multicaster » and « Multicast Controller ». The first one converts a given stream from unicast to multicast, the second one manages the availables multicast resources and dynamically configures the first element. We describe here the technical tracks that were considered and those that were selected, then we describe their implementation.

Résumé

Ce document fait le bilan d'un stage de six mois chez *Orange labs* à Lannion. Ce stage achève le M2 IR (master informatique professionnel, spécialité Ingénierie des Réseaux) que je prépare à l'*ISTIC* de l'université *Rennes 1*. Le double objectif de ce rapport est de décrire le contexte du stage et de rendre compte du travail réalisé.

Sur le fond, un résumé sera fait du contexte actuel de l'utilisation du multicast IP et de ses principales problématiques afin de montrer comment il est – techniquement – possible pour un opérateur de répondre dynamiquement à des besoins de diffusion multicast ponctuels. On peut imaginer des besoins externes aussi bien qu'internes à un opérateur. Sous la forme d'un nouveau service de diffusion par exemple, ou d'un processus de basculement de flux vers le multicast pour économiser les ressources du réseau. Initiée par le projet *Envision*, nous expliquerons en quoi la solution proposée est intéressante pour ce projet et comment elle s'y intègre.

Mon travail durant ce stage a été d'étudier et de mettre au point deux nouveaux éléments que la solution proposée fait intervenir dans le réseau de l'opérateur : le « multicateur » et le « contrôleur multicast ». Le premier transforme un flux unicast donné en un flux multicast, le second gère les ressources multicast disponnibles et configure dynamiquement le premier. Nous montrerons quelles solutions techniques ont été envisagées et lesquelles ont été retenues, puis nous décrirons leur mise en place.

Table des matières

Table des figures **5**

Liste des tableaux **6**

Remerciements **7**

Introduction **8**

1 Contexte du stage **9**

2 Problématique et objectifs du stage **12**
2.1 Problématique 12
2.2 Intégration dans le projet Envision et objectifs du stage 16

3 Réalisation **19**
3.1 Mise au point d'un multicasteur avec GNU/Linux 19
3.1.1 Configuration des Translations d'adresse 20
3.1.2 Routage des paquets multicast 21
3.1.3 Résultats, cheminement des paquets dans le noyau Linux 22
3.2 Conception et développement du contrôleur 24
3.2.1 Conception logicielle 24
3.2.2 État du développement 24
3.3 Extensions et tests 25
3.3.1 Configuration du contrôle des débits 25
3.3.2 Tests fonctionnels et tests de performance 27

Conclusion **29**

Bibliographie **30**

A Annexes techniques **31**
A.1 Diagrammes de conception du contrôleur 32
A.2 Configuration d'un multicasteur sous GNU/Linux 34
A.3 Note technique sur le contrôleur multicast 40

Table des figures

1.1 Répartition des activités dans l'URD MOVI 11

2.1 Multidiffusion IP actuelle avec un contrôle d'accès statique 14
2.2 Notre solution pour la multidiffusion IP « à la demande » (contrôle d'accès statique) 15
2.3 Intégration du multicast à la demande dans le projet Envision 17

3.1 Fonctionnement du multicasteur sur un exemple 19
3.2 Chaînes iptables, filtrage, routage 20
3.3 Cheminement des paquets dans le noyau Linux 23
3.4 Résultat de l'exemple : le multicasteur limite bien le débit à 10Mbit/s 27
3.5 Evolution du débit d'entrée, du débit de sortie et de l'utilisation processeur lors de la monté en charge 28

A.1 Diagramme de déploiement 32
A.2 Diagramme de cas d'utilisation du contrôleur 32
A.3 Diagramme de classes du contrôleur multicast 33

Liste des tableaux

2.1 Solutions retenues pour les différents points du service de diffusion multicast 17

Remerciements

Je remercie d'abord Irène Grosclaude, mon encadrante, qui a guidé le stage avec pédagogie et bienveillance. Le travail technique demandé a ajouté plusieurs cordes à mon arc. Merci à Bertrand Mathieu, responsable du projet Envision pour ses éclaircissements sur le projet et ses finalités. Merci aussi à tous les deux pour votre aide et vos conseils concernant une éventuelle poursuite en thèse.

Merci aux collègues que j'ai beaucoup sollicité pour des questions ou des coups de main pendant six mois, à savoir principalement : Michel, Thomas, Jean-Yves, Jean-Louis, Patrick et Myriam sans oublié Gwendal mon collègue stagiaire de Toulouse.

Merci à toute l'équipe MOVI pour cet accueil et cette bonne ambiance, je dois dire que j'ai appris beaucoup de choses grâce à la pause café du matin :-). Je ne trouve rien à dire quand on me demande ce qui a manqué ou ce qui s'est mal passé : c'est bon signe.

Introduction

À l'issue de la formation de l'ISTIC en ingénierie des réseaux, le stage de six mois a vocation à faire la transition entre le monde universitaire et le monde professionnel en permettant aux étudiants d'acquérir une expérience significative dans leur domaine de compétence. Il constitue une étape décisive pour l'orientation et l'insertion professionnelle. Étant intéressé par l'évolution des réseaux informatiques depuis deux ans, j'ai vu en ce stage le moyen de confirmer mon intérêt pour la recherche et développement (R&D) et d'évaluer mes aptitudes pour ce type de travail. Bien qu'il se soit avéré plus pratique que théorique, plus « développement » que « recherche », ce stage m'a permis de consolider le bagage technique de ma formation et dans le même temps de conforter mon choix d'orientation professionnelle.

Comme on le verra, les objectifs du stage m'ont amené à travailler sur de multiples sujets. Ils ont d'abord été l'occasion de reprendre certaines thématiques des réseaux vues en cours : le multicast IP, la translation d'adresse, le filtrage, le contrôle de trafic ; mais aussi d'utiliser de nouveaux outils : simulation de trafic avec un serveur N2X, configuration de multicast IP sur des routeurs Juniper. Des compétences transverses ont aussi été sollicitées comme le développement logiciel ou la configuration réseau sous GNU/Linux.

Ce rapport est organisé en trois parties. On présentera d'abord le contexte de travail : le laboratoire, les activités, les équipes. On expliquera ensuite la problématique à l'origine du sujet de stage, les objectifs de celui-ci et les solutions envisagées puis retenues pour y répondre. Enfin, nous décrirons la réalisation technique des solutions retenues et leurs résultats.

Chapitre 1

Contexte du stage

Avant de présenter le travail réalisé dans ce stage, il convient de présenter brièvement l'entreprise d'accueil et l'unité de recherche et développement dans laquelle j'ai travaillé sur le site de Lannion. Ma position dans la hiérarchie groupe France Telecom fut la suivante : FT / OLNC / RD / CORE / TPN / MOVI.

France Télécom / Orange labs

France Télécom est une grande entreprise de télécommunications (167 000 personnes) commercialisant trois grande familles de services : les services de communication résidentiels (opérateur), les services de communication personnels (mobiles) et les services de communication d'entreprise. Historiquement issue de l'opérateur de télécommunications national, l'entreprise est fondée en 1988, prend la forme juridique d'une société anonyme (1996) avant d'être privatisée (2004).

Orange Labs, dénommé France Télécom R&D avant janvier 2007, est la division recherche et développement du groupe France Télécom. Elle est intégrée dans l'entité globale OLNC (Orange Labs Networks and Carriers). Orange Labs compte 18 laboratoires sur trois continents et en France environ 3 000 employés sur 9 sites : Issy-les-Moulineaux, Lannion, Grenoble, Caen, Rennes, Sophia Antipolis, La Turbie et Belfort. (sources *Wikipedia [fr]* :[1],[2],[3])

Les activités chez Orange labs se répartissent sur environ un tiers de recherche et deux tiers de développement. On comprend par activitées de recherche les travaux portant sur les nouvelles technologies servant le groupe, et par activités de développement les travaux réalisés pour des clients extérieurs. Le travail y est organisé de façon matricielle, les branches d'Orange labs ont chacune leur spécialité, mais leurs entités peuvent travailler en partenariat sur un certain nombre de projets.

On distingue chez Orange labs 6 Centres de Recherche et Développement (CRD), dont les deux principaux sont RESA : pour les réseaux d'accès et CORE pour le cœur de réseau. Ces deux CRD comptent chacun 450 personnes.

[1] `http://fr.wikipedia.org/wiki/France_telecom`
[2] `http://fr.wikipedia.org/wiki/Orange_Labs`
[3] `http://fr.wikipedia.org/wiki/Orange_(entreprise)`

Core / TPN / MOVI

Regroupant les activités de recherche sur le cœur de réseau, le CRD CORE est divisé en 6 laboratoires, dont TPN pour réseaux de transport et paquets (Transport ant Packet Networks). Ce laboratoire est issu de la fusion de deux groupes qui travaillaient sur les réseaux IP et sur les méthodes de transmissions. Il regroupe 120 personnes. Il a la particularité de travailler ces dernières années à hauteur de 50% en recherche et 50% en développement.

Le laboratoire est lui-même divisé en 7 Unités de Recherche et de Développement (URD) :
- SOAN pour les réseaux optiques et synchronisation
- OPEN pour la qualité de service,
- BATS pour la collecte de données d'architecture et de flux
- RIV pour le routage et VPN
- TRM pour le management des trafics et ressources
- ATA pour la gestion technico-économique (étude des coûts pour les projets)
- MOVI pour le multicast overlay et les infrastructures virtualisées

Aujourd'hui, le laboratoire TPN est en charge des évolutions des réseaux de transport cœur et collecte du groupe France Télécom. Il a trois missions :
- Piloter et réaliser les études pour l'évolution des réseaux de transport, collecte et cœur, mobile et fixe. Il s'agit d'adapter les réseaux aux besoins des applications pour permettre aux divers pays du groupe, de faire face à la croissance des usages et du trafic avec la qualité de service requise.
- Concevoir des solutions ad-hoc robustes, économiques, tirant parti des évolutions (IPV6, Enhanced Packet Core,...). Pour cela, les architectures combinent les technologies de transmission (optiques et hertziennes), de l'IP ou des nouvelles technologies (CDN, P4P, flow management, réseaux véhiculaires, commutation optique, virtualisation réseau)
- Anticiper les évolutions, d'imaginer des solutions en rupture et de concevoir les réseaux de l'Internet du futur.

C'est dans l'URD MOVI (Multicast Overlay and Virtualized Infrastructures) que s'est déroulé mon stage. Cette unité de recherche travail sur trois thèmes :
- Le transport multicast dans les réseaux fixes pour les services des entreprises et les services audiovisuels grand public (5 personnes).
- Le transport des contenus, audiovisuels notamment, dans les infrastructures réseaux déployées au-dessus des réseaux IP : transport pair-à-pair, transport unicast/multicast dans les Content Delivery Networks (4 personnes).
- La virtualisation des réseaux et les évolutions d'architecture des réseaux pour le cloud computing (7 personnes). Domaine de forte croissance.

Le projet Envision

Envision [4] est un projet collaboratif européen qui propose d'améliorer les performances des réseaux overlay en leur permettant de communiquer avec les réseaux physiques sous-jacents des opérateurs. Deux personnes de l'URD MOVI sont affectées à ce projet : Bertrand MATHIEU qui en est le responsable et Irène GROSCLAUDE, ma responsable, qui travaille sur les aspects multicast.

[4] `http://www.envision-project.org/`

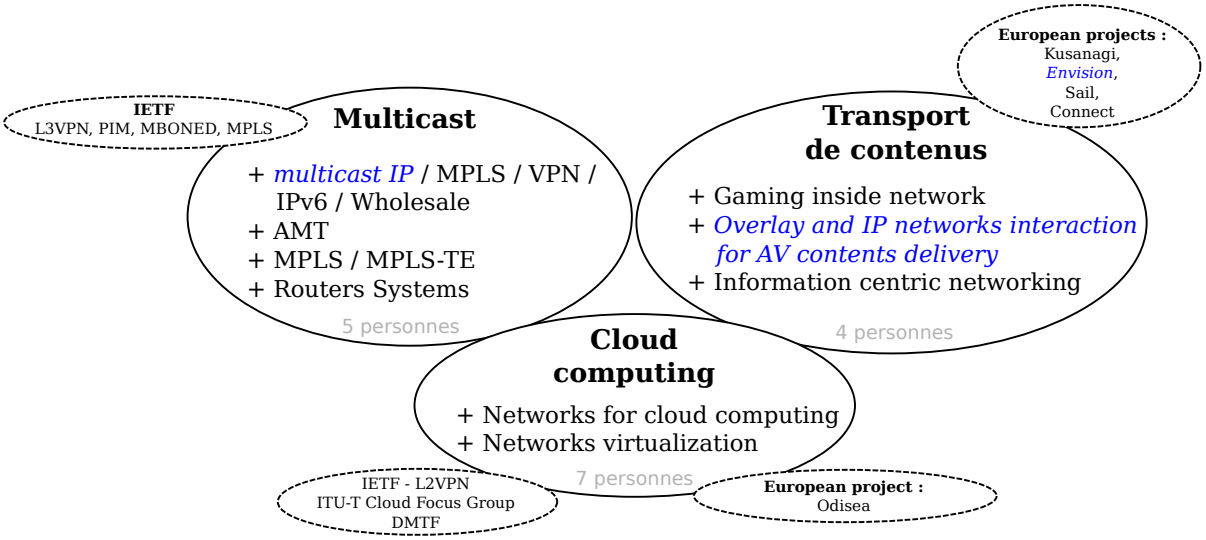


FIG. 1.1 – Répartition des activités dans l'URD MOVI

C'est sur la mise en place d'un service de multicast proposé dans le projet que j'ai travaillé (voir *figure 1.1*). La partie 2.2 explique comment mon travail s'intègre au projet *Envision*.

Chapitre 2

Problématique et objectifs du stage

2.1 Problématique

Le multicast IP est considéré comme un mécanisme efficace pour diffuser du contenu à grande échelle sur Internet. Il permet principalement de préserver la bande passante en évitant la duplication du contenu sur les liens. Il permet par exemple de diffuser un flux à haut débit vers de nombreux utilisateurs sans surcharger un réseau IP. À l'heure actuelle, si le multicast IP est souvent utilisé sur les réseaux universitaires, les opérateurs eux l'utilisent principalement pour leurs chaînes TV.

Situation des opérateurs concernant le multicast IP

Il existe deux modes de diffusion multicast : « any source multicast » et « specific source multicast ». Ces deux modes se basent sur la construction d'arbres de diffusion permettant l'acheminement des paquets entre une source et un groupe d'abonnés.

En ASM, un client s'abonne à un groupe multicast sans préciser de source particulière, il recevra tous les flux sans distinction sur sa provenance (on parle de communication « plusieurs-à-plusieurs »). On construira dans ce mode un arbre de diffusion par groupe multicast. Comme cet arbre peut-être partagé par plusieurs émetteurs, un point de rendez-vous devra être fixé pour définir le nœud racine de l'arbre.

En SSM, un client s'abonne à un groupe multicast et à une source précise (on parle de communication « un-à-plusieurs »). Ainsi dans ce mode l'arbre multicast relie une source à ses destinations sans point de rendez-vous et chaque source possède son arbre de diffusion pour un groupe donné.

Les principaux freins techniques de l'ouverture du multicast par les opérateurs sont les problèmes liés à la sécurité. En ASM comme en SSM, la multidiffusion présente des risques au niveau infrastructure et au niveau applicatif. Au niveau de l'infrastructure, une station n'étant pas limitée par le nombre de flux auquelle elle s'abonne, la surcharge sur des liens est possible ou des attaques de type dénis de services si on ne contrôle pas les abonnements on niveau du réseau d'accès.

Au niveau applicatif, la confidentialité ne peut pas être assurée car une source ne connaît pas ses destinataires.

En utilisant SSM on évite des risques qui sont propres à ASM, à savoir par exemple la surcharge facile du réseau au niveau du point de rendez-vous et on limite la possiblité de recevoir de paquets de sources inconnue (l'IP spoofing plus difficile qu'en unicast). C'est pourquoi les opérateurs utilisent

la diffusion multicast SSM (PIM-SM SSM) en contrôlant l'accès des flux de données multicast à leur réseau et en contrôlant les abonnements. Ils autorisent certains flux et certains abonnements, c'est ainsi que les flux vidéo des chaînes TV sont diffusés.

Multicast IP « à la demande »

Il semble particulièrement intéressant d'utiliser le multicast IP pour diffuser ponctuellement du contenu live étant donné la proportion que peuvent prendre certaines diffusions vidéo évenementielles. On peut imaginer qu'un service proposé par un opérateur qui permette d'utiliser les ressources multicast IP pour une période définie serait dans le même temps intéressant pour cet opérateur qui ne surchargerait pas son réseau.

Si un opérateur voulait mettre en place un tel système, il devrait – pour les raisons de sécurité décrites plus haut – autoriser au moment voulu l'accès multicast des flux voulus. Comme il le fait actuellement pour les flux TV, mais dynamiquement. Or les opérateurs ne peuvent pas toujours compter sur une configuration dynamique de leur équipements d'accès (MSAN) car certains équipements d'anciennes génération n'intègrent pas les fonctions nécessaires.

Une solution possible dans cette situation est de définir un ensemble de flux multicast (Si,Gi) autorisés à circulés dans le réseau (les flux de données et les abonnements), puis de transformer les flux – initialement unicast – que l'on souhaite diffuser en flux multicast en choisissant dans le pool d'adresses (Si,Gi) autorisées.

Pour déployer cette solution, on introduit deux nouvelles entités dans le réseau de l'opérateur :

1. Le *multicasteur* (MC) sera un noeud capable de transformer un flux unicast en un flux multicast via une translation des adresses source et destination (double NAT). Cette translation sera configurée dynamiquement par le contrôleur au moment voulu
2. Le *contrôleur multicast* (MCC) sera responsable de l'allocation et de la désallocation des états multicast (Si,Gi) au gré des demandes et en fonction : des ressources autorisées (accès autorisé sur les MSAN) et disponibles à un instant donné. C'est cette entité qui configurera au moment voulu l'un des MC qu'il contrôle.

Fonctionnement sur un exemple

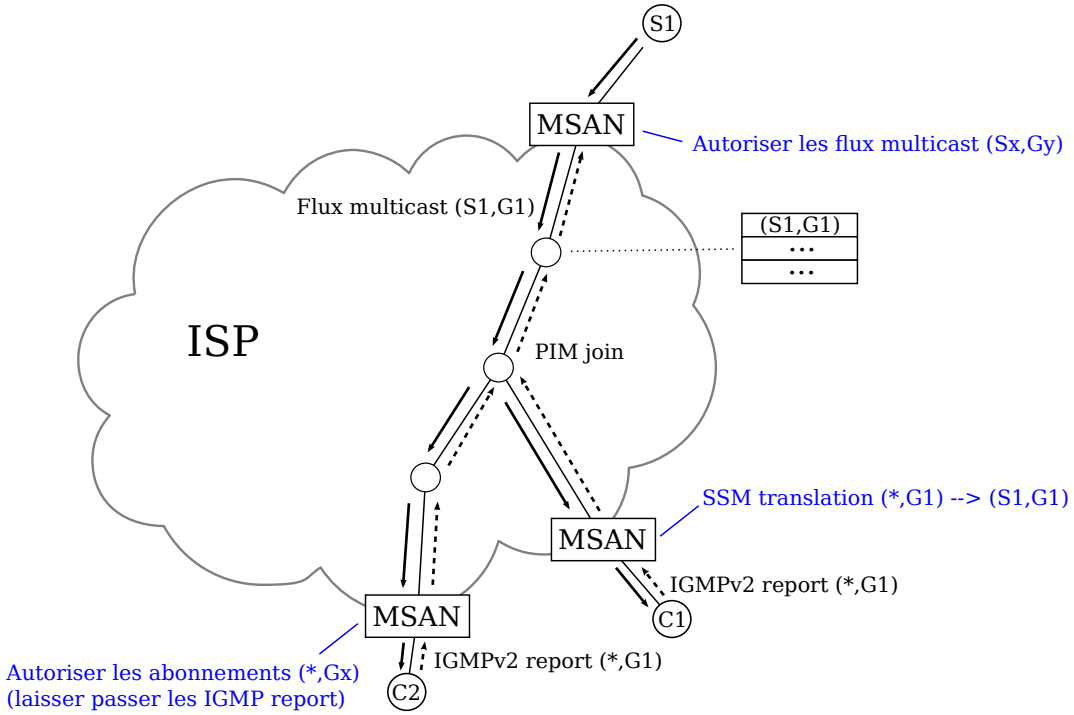


FIG. 2.1 – Multidiffusion IP actuelle avec un contrôle d'accès statique

Multidiffusion IP actuelle avec un contrôle d'accès statique (*figure 2.1*) Pour diffuser avec PIM-SM SSM sur le groupe G1 depuis la source S1, l'ISP va devoir autoriser l'accès au trafic (S1,G1) statiquement et autoriser les abonnements IGMP (S1,G1) correspondants. Dans le cas de *Orange*, où IGMPv2 est utilisé, les messages abonnements du type *(*,G1)* sont manipulés au niveau des routeurs d'accès pour ajouter une source associée au groupe et transformer l'abonnement en (S1,G1). Cette transformation SSM (SSM translation) qui associe une source à un groupe est elle aussi statique

Comme on le voit, pour changer la source d'un flux multicast, le contrôle d'accès doit être reconfiguré sur tous les équipements d'accès pour que la diffusion se fasse de bout en bout, ce qui n'est pas imaginable statiquement.

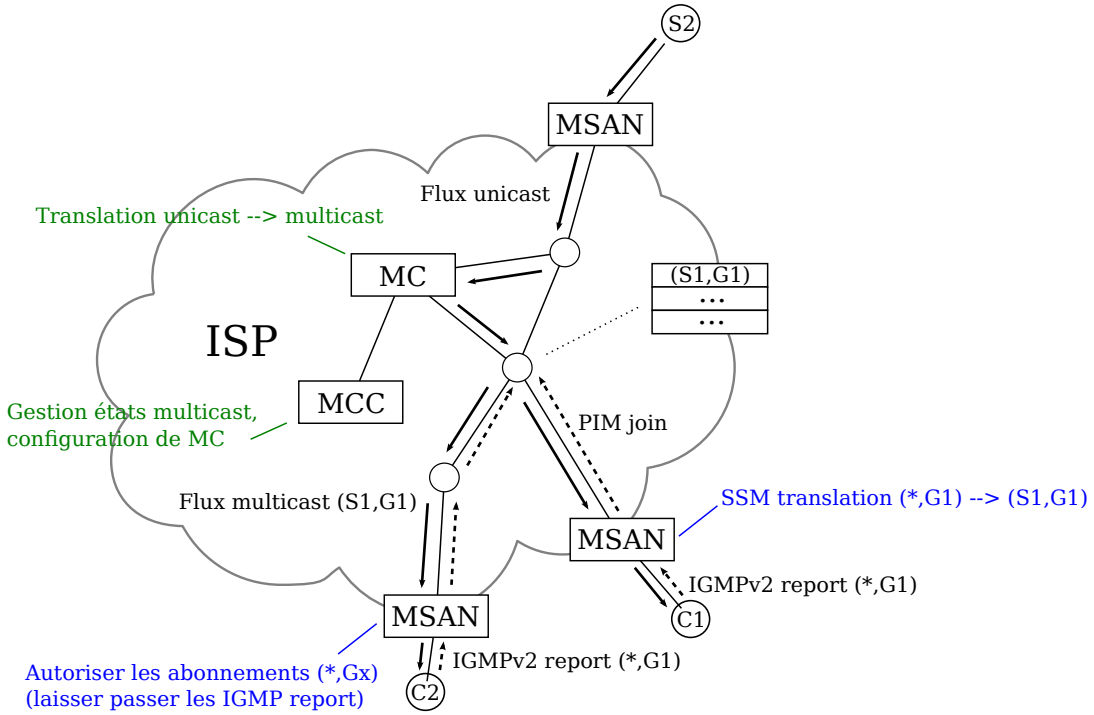


FIG. 2.2 – Notre solution pour la multidiffusion IP « à la demande » (contrôle d'accès statique)

Notre solution pour la multidiffusion IP « à la demande » (*figure 2.2*)

1. On autorise un certain nombre de sources et groupes à ête utilisés (S,G). C'est à dire on autorise les abonnements et on autorise leur accès au réseau statiquement sur les équipements d'accès (MSANs). On oubliera pas la SSM translation sur les routeurs d'accès pour les opérateurs qui comme Orange utilise IGMPv2
2. On imagine un service réservable à l'avance.L'utilisateur du service multicast qui souhaite diffuser va adresser une requête au MCC. Ce dernier va lui dire si il peut lui allouer des ressources (un couple S,G) conformes à sa requête parmis les ressources disponibles configurées sur les MSANs. Il va lui dire quelle ressources il lui seront allouées si c'est possible et lui préciser l'adresse IP du MC
3. Au moment voulu, le MCC va configurer le MC pour lui ajouter la translation permettant la diffusion entendue avec l'utilisateur
4. Le serveur de flux de l'utilisateur va envoyer le flux unicast à diffuser à destination du MC

2.2 Intégration dans le projet Envision et objectifs du stage

C'est dans le projet *Envision* [MGE+11] que s'intègre le service de multicast à la demande. Ce projet, qui a pour objectif de faire communiquer les réseaux Overlay (exemple : réseau Pair à Pair) et les réseaux physiques des opérateurs en spécifiant une interface de communication (CINA), doit permettre aux réseaux Overlay d'améliorer leurs performances mais aussi d'accéder à des nouveaux services. La connaissance du réseau physique donne notamment la possibilité à un réseau overlay de tirer profit du multicast IP.

Voici ce que dit en substance le délivrable d3.1 du projet au sujet du multicast :

> Le multicast au niveau applicatif (ALM) a été proposé comme une alternative au multicast IP répondant au problème de diffusion de bout en bout expliqué ci-avant. Dans les solutions ALM, les paquets sont transmits d'un nœud à l'autre en unicast en suivant un arbre de diffusion. Mais les solutions ne tenant pas compte du réseau physique sout-jacent, la topologie des arbres, malgré les optimisations proposées par certaines études [RHKS01] ne peut pas être optimale. Le trafic pouvant être amené à passer plusieurs fois par les même liens, les bénéfices de cette technique sont sans comparaison avec ceux du multicast IP natif.
>
> Pour le projet Envision, il est proposé d'utiliser le multicast hybride pour permettre aux opérateurs de tirer profit du multicast lorsque c'est possible, et de relayer la diffusion du contenu via les liens unicast de l'overlay pour les zones du réseau ou le multicast IP n'est pas disponible. Il s'agira pour cela de définir une interface entre le réseau d'un opérateur et le réseau overlay (cette interface est appelée CINA dans Envision). Il s'agira également de mettre en place, côté opérateur, un système capable de transformer un trafic IP unicast en trafic IP multicast et inversement. On nommera ces systèmes multicasteur et unicasteur. Ces systèmes devront être configurables dynamiquement et à l'avance depuis une entité de contrôle que l'on appelera multicast controlleur qui sera à l'écoute des requêtes du ou des réseaux overlay grâce à l'interface CINA.

Comme on le voit c'est dans un cadre bien précis que ce système de « multicast à la demande » a été initié, mais il peut tout à fait être utilisé indépendamment du projet *Envision*

Objectif

L'objectif général du stage est resté le même durant les six mois : il s'est agit de définir le fonctionnement technique du système de « multicast à la demande » décrit à la fin de la partie 2.1 et de les mettre en place en vue d'un démonstrateur. Quelle technologie utiliser pour le multicasteur ? pour le multicast contrôleur ? comment les faire communiquer ?

La *figure A.1* illustre le déploiement des entités du système de multicast à la demande et leur intégration au projet *Envision*. Le *tableau 2.1* récapitule les solutions retenues (en vert) à chaque niveau et précise les parties réalisées pendant le stage (en gras).

Solutions envisagées, solution retenue

Comme on le voit sur le *tableau 2.1*, certaines solutions n'ont pas été retenues. La première piste pour la configuration dynamique d'un NAT unicast vers multicast a été l'association de *netconf* et

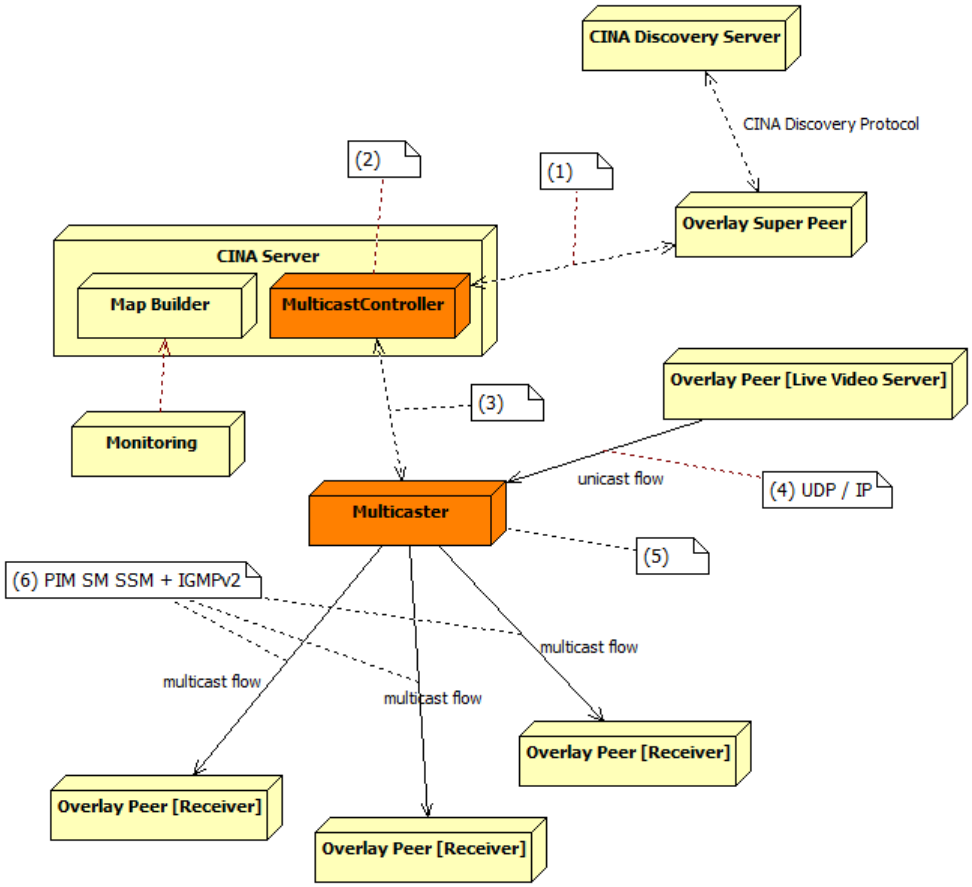


FIG. 2.3 – Intégration du multicast à la demande dans le projet Envision

	Rôle	***Solutions***	
(1)	Overlay ↔ Controlleur	Protocole CINA	Madcap étendu
(2)	Controlleur	**Application Java**	**Script Python**
(3)	Controlleur ↔ Multicasteur	SSH	Netconf
(4)	Transport/Routage unicast	*/IP	
(5)	Multicasteur	**GNU/Linux** (1)	**Junos-SDK** (2)
(6)	Routage multicast	PIM SM SSM + IGMPv2	

TAB. 2.1 – Solutions retenues pour les différents points du service de diffusion multicast

de la translation d'adresse proposée par les routeurs. En effet, si un routeur pouvait faire office de multicasteur en proposant la translation d'adresse vers le multicast, *netconf* pourrait être utilisé par le contrôleur multicast pour configurer au moment voulu le NAT du routeur. Après quelques recherches sur la translation d'adresse sur les routeurs de plusieurs constructeurs (Cisco, Alcatel et Juniper) j'ai constaté – comme je m'y attendais – que aucun ne permettait de tranformer un flux unicast en multicast.

La version GNU/Linux du multicasteur avec la comunication en SSH avec le contrôleur représente une première étape du service de multicast à la demande à des fins de démonstration. Cette solution a été choisie car elle répond à la contrainte de n'utiliser que des outils open source et gratuits pour le démonstrateur du projet. Comme ses résultats ont été positifs, le choix a été fait d'améliorer cette solution plutôt que de débuter la version final du système.

La version final consiste comme le titre du stage le suggère à développer les fonctionalités du multicasteur dans des routeurs ouverts. Ce type de développement était envisagé avec le SDK (Software Development Kit) proposé par *Juniper*

Le développement du contrôleur multicast et la technologie qu'il utilise est indépendant des ces solutions pour le multicasteur. Il d'abord commencé à être développé en Python, puis nous avons opté pour Java pour faciliter la prise en main et l'intégration du MCC au serveur CINA.

Chapitre 3

Réalisation

3.1 Mise au point d'un multicasteur avec GNU/Linux

Nous allons montrer ici comment configurer une station GNU/Linux pour transformer un flux unicast en flux multicast. On combinera pour cela la translation d'adresse (NAT) et le filtrage avec *iptables* et le routage multicast statique des paquets résultants de la translation avec *smcroute*.

Prenons pour cela l'exemple de la translation illustrée par la *figure 3.5*. Le serveur de diffusion émet les paquets à destination du multicasteur et ce dernier est chargé de transformer le flux unicast en un flux multicast vers un routeur multicast en devenant la source du groupe vis à vis du domaine multicast (association (S,G) dans PIM SM SSM). On considérera dans cet exemple que les flux unicast et multicast entrent et sortent par la même interface du multicasteur.

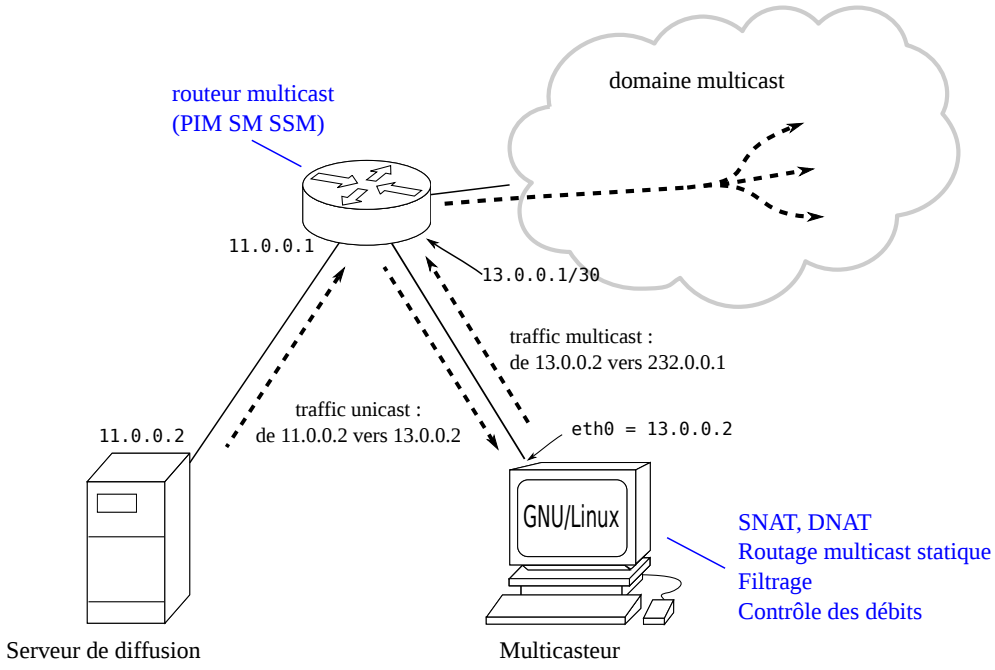


FIG. 3.1 – Fonctionnement du multicasteur sur un exemple

Pré-requis Pour qu'une station GNU/Linux se comporte comme un routeur, on commence par activer la transmission des paquets IP :

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

On vérifie aussi le support du multicast par le noyau :

```
# cat /boot/config-xxxx | grep MULTICAST
# CONFIG_IP_MULTICAST=y
# cat /boot/config-xxxx | grep CONFIG_IP_MROUTE
# CONFIG_IP_MROUTE=y
```

3.1.1 Configuration des Translations d'adresse

Pour configurer un multicasteur, on peut utiliser iptables sous GNU/Linux – qui est installé de base sur la plupart des distributions – pour faire la translation d'adresses (NAT) ainsi du filtrage pour plus de sécurité. Il s'agit de configurer un NAT double (DNAT + SNAT) sur les adresses de destination et de source respectivement lors des chaînes de *prérouting* et de *postrouting*.

Dans la chaîne de prérouting on transcrit l'adresse de destination (DNAT) de l'adresse du multicasteur vers une adresse IP multicast. Ce DNAT entraîne, après le routage, le non passage dans la chaine *input*, on reste dans l'espace noyau et on passe dans les chaînes forward et postrouting. (voir *figure 3.2*)

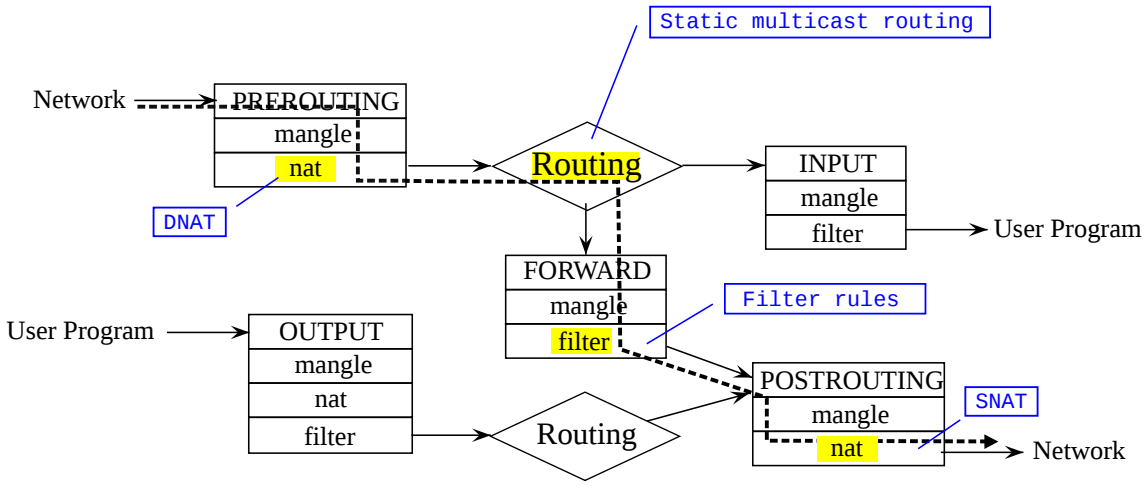


FIG. 3.2 – Chaînes iptables, filtrage, routage

Commandes de NAT correspondantes pour l'exemple

```
# iptables -t nat -A PREROUTING
    -s 11.0.0.2
    -d 13.0.0.2 --destination-port 200
    -j DNAT --to-destination 232.0.0.1
# iptables -t nat -A POSTROUTING
```

```
    -s 11.0.0.2
    -d 232.0.0.1 --destination-port 200
    -j SNAT --to-source 13.0.0.2
```

Filtrage des paquets On bloque tout par défaut et on ajoute des exceptions

```
# iptables -P FORWARD DROP
# iptables -t filter -A FORWARD
    -s 11.0.0.2
    -d 232.0.0.1 --destination-port 200
    -j ACCEPT
```

3.1.2 Routage des paquets multicast

Une fois la transation d'adresse de destination a été réalisée dans la chaîne de *prérouting*, il faut router les paquets considérés dès lors par le noyau comme des paquets multicast. On va donc devoir insérer dans la table de routage multicast l'enregistrement correspondant à notre flux. Sous GNU/Linux plusieurs solutions sont envisageables pour faire cela, nous présentons ci-après les solutions qui ont été enviseagées et pourquoi nous avons retenu *smcroute*.

Solution 1 - la commande ip mroute La commande *ip mroute* ne prend pas en charge l'ajout ou la supression de routes multicast. Extrait du manuel de la commande *ip* : (`http://linux.die.net/man/8/ip`)

«Due to the limitations of the current interface to the multicast routing engine, it is impossible to change mroute objects administratively, so we may only display them. This limitation will be removed in the future»

⇒ ***La solution ne convient pas***

Solution 2 - Implémentations de PIM sous Linux (pimd, mrouted, quagga, xorp)

Pour gérer les routes multicast, on peut considérer notre station GNU/Linux comme un routeur multicast. Pour qu'il se comporte en conséquence plusieurs configurations sont possibles : le configurer avec un démon de routage comme *mrouted* ou *pimd* ou bien installer un routeur logiciel comme Quagga ou XORP qui implémentent PIM SM.

Cependant, si on peut voir le multicasteur comme un routeur multicast au sens où il route des paquets multicast, il est dans le même temps la – nouvelle – source multicast vis à vis du domaine multicast ce qui fait du multicasteur une entité particulière que les implémentations des protocoles de routages énoncés ne vont pas configurer correctement. En effet un démon comme pimd ne va pas pouvoir gérer les routes multicast de sorte que l'entité source soit lui même ou bien une source que l'on voudra choisir : tout est fait automatiquement.

⇒ ***Les solutions pimd et mrouted ne conviennent pas***
⇒ ***Les solutions routeurs logicielles n'ont pas été testées***

Solution 3 - Implémentation pour le routage multicast static (smcroute)

L'outil *smcroute* permet de faire le travail de *pimd* ou *mrouted*, c'est à dire gérer la table de routage multicast du noyau Linux, mais de façon statique. On va pouvoir ajouter manuellement les routes multicast statiques que l'on veut. Dans notre exemple, le cas d'utilisation est particulier car on reçoit les paquets unicast et on envoi le flux multicast sur la même interface.

Ajout de la route multicast statique de l'exemple

```
# smcroute -a eth0 11.0.0.2 232.0.0.1 eth0
```

On peut vérifier le cache de la table de routage multicast du noyau avec ip mroute :

```
# ip mroute show
(11.0.0.2, 232.0.0.1)      Iif: eth0      Oifs: eth0
```

⇒ ***La solution convient et fonctionne***

3.1.3 Résultats, cheminement des paquets dans le noyau Linux

Résultat La combinaison *iptables/smcroute* fonctionne bien sur notre exemple, en monitorant l'interface eth0 du multicasteur avec *tshark*, on obtient la capture suivante :

```
l-otoquar-dev:/home/envision# tshark -i eth0 -c 20
Capturing on eth0
   0.000000      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000006      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000070      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000079      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000138      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000146      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000207      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000213      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000274      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000280      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000344      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000350      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000412      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000419      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000486      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000492      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000550      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000556      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
   0.000692      11.0.0.2 -> 13.0.0.2    UDP Source port: src    Destination port: src
   0.000701      13.0.0.2 -> 232.0.0.1   UDP Source port: src    Destination port: src
3 packets dropped
20 packets captured
```

Cheminement des paquets Les paquets vont passer par plusieurs étapes entre leur entrée et leur sortie :

1. ils vont subirent une première translation de leur adresse de destination dans la chaîne de pré-routage qui va en faire des paquets multicast. On constate que les adresses MAC des trames ethernet de ces paquets sont bien préfixé par *01 :00 :5E* pour correspondre au multicast IPv4 à l'issue de la translation.
2. ils seront routés selon les enregistrement présents dans la table de routage multicast remplie avec *smcroute*
3. ils seront filtrés de sorte que seuls les flux translatés soit transmits afin de limiter les problèmes de sécurité
4. ils vont enfin subir une autre translation d'adresse, sur leur source cette fois. Cela permet dans notre exemple de faire passer le multicasteur pour la source du flux vis à vis du domaine multicast puisque on donnera comme nouvelle source sont interface de sortie eth0. On peut cependant définir n'importe quelle adresse unicast IPv4 comme la source du flux.

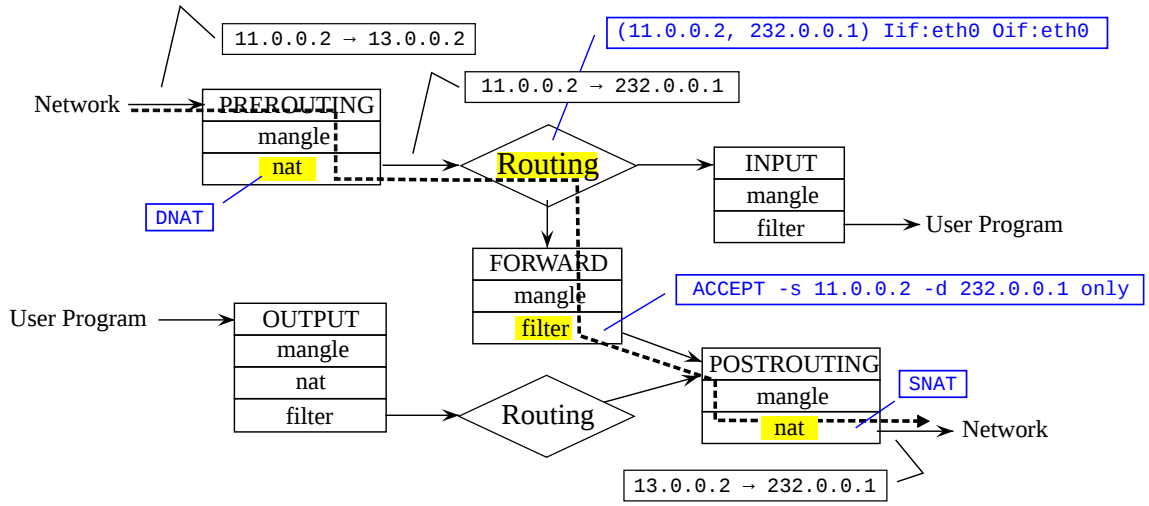


FIG. 3.3 – Cheminement des paquets dans le noyau Linux

3.2 Conception et développement du contrôleur

3.2.1 Conception logicielle

Fonctions générales Le contrôleur multicast a deux fonctions principales :

1. il doit gérer l'allocation des ressources multicast qui c'est à dire à un pool de couples (S,G) qui correspondent aux flux de données multicast autorisés à circuler dans le réseau. Cette une gestion dans le temps des ressources via des créneaux horaires
2. il doit configurer les multicasteurs selon les allocations qu'il a autorisé au moment voulu. Pour ça, il va (dans la version actuelle avec GNU/Linux) établir une connection SSH et exécuter une succession de commandes linux qui configureront la translation du nouveau flux.

Intégration dans Envision Dans le cadre d'un service de multicast, un client qui souhaite diffuser un flux multicast va découvrir un contrôleur en charge des ressources multicast via l'interface CINA (Cf CINA Discovery Protocol). Il lui transmettra ensuite une requête de réservation multicast avec le protocol CINA. S'il reste des ressources disponibles, le multicast contrôleur va lui transmettre l'adresse IP du multicasteur auquel envoyer le flux unicast et le couple (S,G) auquel s'abonner. C'est ensuite au réseau overlay de se charger d'informer la pair overlay qui souhaite diffuser d'envoyer son flux en unicast à l'adresse du multicasteur et d'informer les pairs overlay de l'abonnement à (S,G).

Les principaux diagrammes de la conception logicielle sont fournis en annexe à la fin du rapport (A.1) pour donner une idée de son fonctionnement et une note technique précisant les librairies utilisées et les références est en annexe A.3.

3.2.2 État du développement

Le contrôleur multicast est encore en développement. Il a vocation à être intégré à une brique applicative plus importante de Envision. Dans son état actuel il est utilisable pour des tests avec un menu textuel.

Travail fait Le contrôleur multicast est pour le moment assez basique par rapport aux fonctions envisagées, il ne fait que l'essentiel. Il peut configurer automatiquement à l'avance la translation d'un flux unicast vers multicast. On lui précise la source du flux, le port de destination, la date de début et la date de fin et il choisi, – pour l'instant arbitrairement – un multicasteur à configurer et un groupe multicast.

Travail restant La suite du développement va consister à : faire le lien entre l'interprêteur du protocole CINA utilisé pour les requêtes, gérer les conflits d'allocation des créneaux horaires, récupérer les informations de monitoring du réseau (le service de monitoring n'est pas terminé). Choisir les ressources à utiliser intelligemment : le multicasteur et le couple (S,G). Il faudrait également gérer la connexion SSH aux multicasteurs avec des clés plutôt que des login/mot de pass et modifier le contrôleur en conséquence.

3.3 Extensions et tests

3.3.1 Configuration du contrôle des débits

Solution 1- Extension netfilter Avec les patch fuzzy des extensions de *netfilter* installées avec le patch 'Patch-O-Matic' ; *iptables* propose des options permettant de matcher les paquets selon le débit de leur flux. À noter que pour cela il faut recompiler le noyau Linux.

L'idée de base est que l'on donne au match deux paramètres pour décrire l'intervalle de filtrage.
- Quand le débit des paquets est en dessous de 'lower-limit' la règle ne matchera jamais.
- Entre 'lower-limit' et 'upper-limit', plus le débit des paquets s'approche de 'upper-limit' et plus le module matchera.
- Finalement, quand le débit des paquets atteint 'upper-limit', le module atteint son maximum, et matche les paquets à 99%.

Par exemple, pour éviter les Denials Of Service, on peut utiliser la règle suivante :

```
# iptables -A INPUT -m fuzzy --lower-limit 100 --upper-limit 1000 -j REJECT
```

⇒ Problème : Le développement de ce patch n'est plus actif. La dernière version date de 2003 et elle est introuvable pour le téléchargement.

Solution 2 - Contrôle de trafic avec le paquet ip route 2

C'est avec la commande *tc* que l'on va mettre en place le contrôle des trafics sortants. Le système de contrôle de trafic sous Linux permet de mettre en place des mécanismes évolués pour la limitation ou l' « agencement » du trafic (shaping). On a accès à ne nombreux algorithmes de queuing plus ou moins complexes avec des cas d'utilisations particulier pour chacun d'eux.

On peux, pour le cas basique qui nous intéresse – c'est à dire borner les trafics sortant selon leur groupe multicast et leur port de destination –, utiliser plusieurs types d'ordonnancement (qdisc) : le Class Based Queuing (CBQ) ou le Hierarchical Token Bucket (HTB). On choisira le HTB qui est, dans notre cas d'utilisation, le plus simple à comprendre et à mettre en place (moins de commandes pour faire la même chose).

Le principe de l'algorithme HTB (« Seau à jetons hiérarchique ») est de limiter le débit maximum d'un flux de données. Il ne doit pas être confondu avec l'algorithme du seau percé, qui permet de fluidifier un trafic non régulier (« en rafales ») en jouant un rôle de tampon et en permettant de réguler son débit.

Son fonctionnement est simple : Un trafic est associé à un seau à jetons, et les paquets appartenant à ce trafic utilise des jetons du sceau lors de leur passage. Périodiquement, des jetons sont ajoutés dans le sceau. Selon la vitesse de remplissage du seau, on a à tout moment un trafic qui ne peu excéder un certain débit.

La commande *tc* du paquet *ip route 2* permet :

- D'associer un ordonnanceur de type HTB à une interface : [root qdsic]
- De définir des classes (de trafic) filles de cet ordonnanceur [child class]
- Ces classes filles pouvant elles même contenir d'autres classes filles, être des feuilles [leaf], ou encore contenir d'autre ordonnanceurs [qdisc]
- Les classes et qdisc sont identifiées par deux numéros séparés par ' :'

Dans notre cas, nous avons besoin de limiter le débit de chaque flux que l'on transforme (on identifiera un flux avec son port de destination). On définira donc une *root qdisc* que l'on associera à l'interface de sortie du multicasteur (dans notre exemple eth0). On va définir une classe de traffic pour chaque flux (*child class*) définie avec un débit maximum et les paquets seront classifiés avec des *filtres* selon le port de destination. On ajoutera des *qdsic* de type file SFQ pour répartir équitablement l'utilisation de la bande passante sortante entre les différents flux.

On choisira de ne pas permettre l'emprunt de la bande passante pour les classes de même niveau (« borrowing »), on aura donc des débits garantis mais qui ne pourront pas être dépassés même s'il reste de la bande passante de la classe mère non utilisée.

Si par exemple on veut configurer la hiérarchie suivante :

```
eth0                          1:            root qdisc
                             / | \
                            /  |  \
                           /   |   \
multicast traffics       1:1  1:2  1:3      classes (rates = 10,20,70Mbps)
classified by dport      |    |    |
                         10:  20:  30:      qdiscs
                         sfq  sfq  sfq
                          0    1    2
```

on utilisera ces commandes :

```
# root qdisc sur eth0
tc qdisc add dev eth0 root handle 1: htb

# classes
tc class add dev eth0 parent 1: classid 1:1 htb rate 10mbit ceil 10mbit
tc class add dev eth0 parent 1: classid 1:2 htb rate 20mbit ceil 20mbit
tc class add dev eth0 parent 1: classid 1:3 htb rate 70mbit ceil 70mbit

# filters
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip dport 200 0xffff flowid 1:1
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip dport 223 0xffff flowid 1:2
tc filter add dev eth0 protocol ip parent 1:0 prio 1 u32 match ip dport 225 0xffff flowid 1:3

# qdisc
tc qdisc add dev eth0 parent 1:1 handle 10: sfq perturb 10
tc qdisc add dev eth0 parent 1:2 handle 20: sfq perturb 10
tc qdisc add dev eth0 parent 1:3 handle 30: sfq perturb 10
```

Résultat

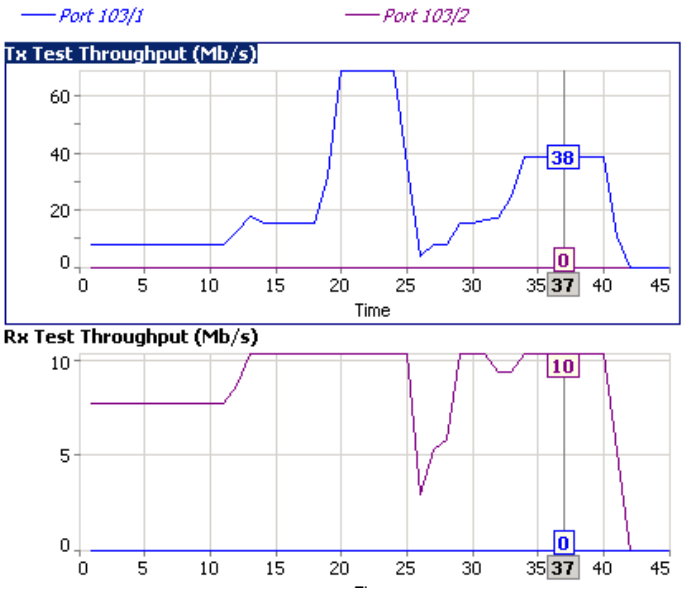


FIG. 3.4 – Résultat de l'exemple : le multicasteur limite bien le débit à 10Mbit/s

3.3.2 Tests fonctionnels et tests de performance

Scénario 1 Ce scénario de test correspond à la configuration décrite dans l'annexe A2.

Objectifs du test : Utilisation de deux interfaces distcinctes pour l'entrée unicast et la sortie multicast.

Description : Envoi d'un flux unicast de 11.0.0.2 vers 14.0.0.2 (eth0 du MC). NAT pour donner un flux de 15.0.0.2 vers 232.0.0.1. Retransmission par l'interface de sortie eth1du MC (15.0.0.2).

Résultat : OK

Scénario 2

Objectifs du test : Ce deuxième scénario doit permettre de vérifier plusieurs choses :

1. translation de plusieurs flux en même temps
2. NAT différents de plusieurs flux ayant la même source (NAT selon le port de destination)
3. débits maximum différents pour chaque flux
4. test d'interruption de la transmission lors de la configuration du deuxième flux
5. test d'ajout d'une configuration avec de la charge
6. test de charge maximum

Description : Envoi d'un flux unicast de 11.0.0.2 vers 14.0.0.2 (eth0 du MC) avec le dport 200. NAT pour donner un flux de 15.0.0.2 vers 232.0.0.1. Retransmission par l'interface de sortie eth1 du multicasteur (15.0.0.2). Envois d'un deuxième flux unicast de 11.0.0.2 vers 14.0.0.2 avec le dport 300. NAT pour donner un flux de 15.0.0.2 vers 232.0.0.2. Retransmission par l'interface de sortie eth1 du MC (15.0.0.2)

Résultats :

1. OK
2. OK
3. OK
4. OK (pas d'interruption)
5. OK (mais configuration plus longue à s'établir)
6. OK (le multicasteur ne tombe pas quelque soit le débit reçut en entrée)

Performances Une station GNU/Linux est bien sûr contre-indiquée pour faire transiter des débits élevés. Le processeur se chargeant ici de la translation d'adresses et du filtrage en plus du routage multicast alors qu'il n'est pas conçu pour ce type d'opérations comme l'est un routeur. C'est une des raisons pour laquelle la version finale avec un service développé sur des routeurs avec le Junos-SDK est indispensable pour un déploiement en conditions réelles.

Scores des deux stations utilisées durant le stage :
- *otoquar* : débit maximum = **77 Mbps**
- *pc-envision* : débit maximum = **46 Mbps**

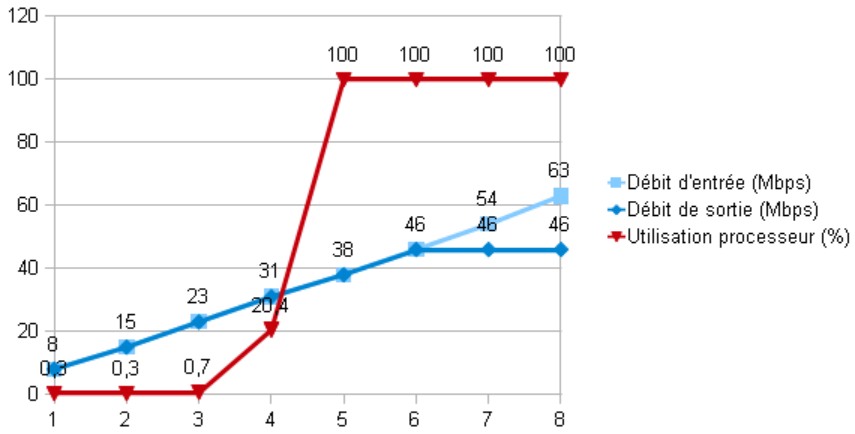


FIG. 3.5 – Evolution du débit d'entrée, du débit de sortie et de l'utilisation processeur lors de la monté en charge

Conclusions et perspectives

Ce stage m'a été profitable sur plusieurs plans. D'abord, cette expérience de six mois m'a fait approfondir certains points de ma formation en ingénierie des réseaux, surtout sur le plan pratique. J'ai travaillé sur la configuration de routeurs Juniper, j'ai pris en main un simulteur de trafic (Serveur N2X) pour faire des tests sur la plate-forme et j'ai surtout travaillé sur une station GNU/Linux utilisée comme un routeur en configurant : du routage multicast, du filtrage, de la translation d'adresse et du contrôle de trafic.

Sur un plan plus théorique, pour comprendre les enjeux de mon sujet de stage il m'a fallut comprendre la situation d'un opérateur de télécommunication vis à vis de la multidiffusion et quelle problématique elle pose, ce qui m'a permi de voir ou revoir pas mal de choses.

Le deuxième type de travail réalisé fut la conception et le développement logiciel a le contrôleur multicast, ce qui m'a permis de valoriser ma double formation logicielle/réseaux. Le temps a manqué pour développer un service sur un routeur Juniper avec le SDK, ce dernier à néamoins été pris en main à travers l'installation et le déploiement de quelques exemples et me donne une assez bonne idée de ce qui peut-être fait.

Je peux dire à l'issue de ces six mois être responsable de la mise au point complète d'une « brique » du projet Envision, ce qui est assez gratifiant même s'il s'agit d'une version de démonstration. Ayant apprécié le fond et la forme du travail en R&D, je suis candidat pour une thèse proposée par le laboratoire TPN sur le réseaux de transport. Je pense orienter mon avenir professionnel vers la recherche et developpment quel que soit le résultat de cette candidature.

Bibliographie

[MGE+11] Bertrand Mathieu, Irène Grosclaude, Selim Ellouze, Yannick Carlinet, Pierre Paris, Valery Bastide, and Emile Stephan. *D3.1, Initial Specification of the ENVISION Interface, Network Monitoring and Network Optimisation Functions.* 2011.

[Ras07] Michael Rash. *Linux firewalls : attack detection and response with iptables, psad, and fwsnor.* No Starch Press, 2007.

[RHKS01] Sylvia Ratnasamy, Mark Handley, Richard Karp, and Scott Shenker. *Application-Level Multicast Using Content-Addressable Networks.* Springer Berlin / Heidelberg, 2001.

Annexe A

Annexes techniques

A.1 Diagrammes de conception du contrôleur

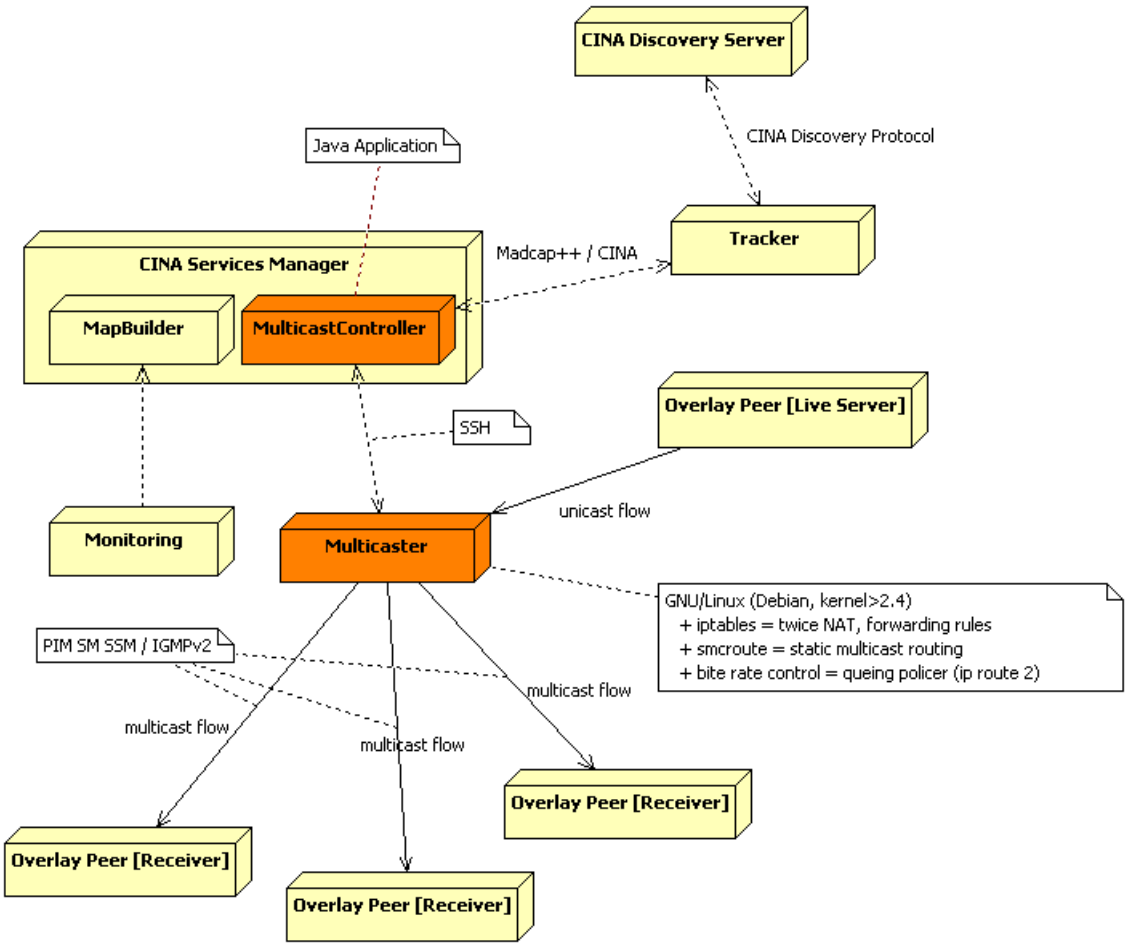


Fig. A.1 – Diagramme de déploiement

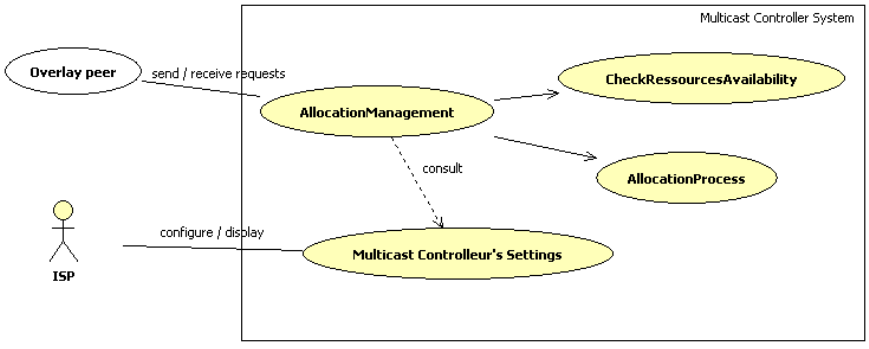


Fig. A.2 – Diagramme de cas d'utilisation du contrôleur

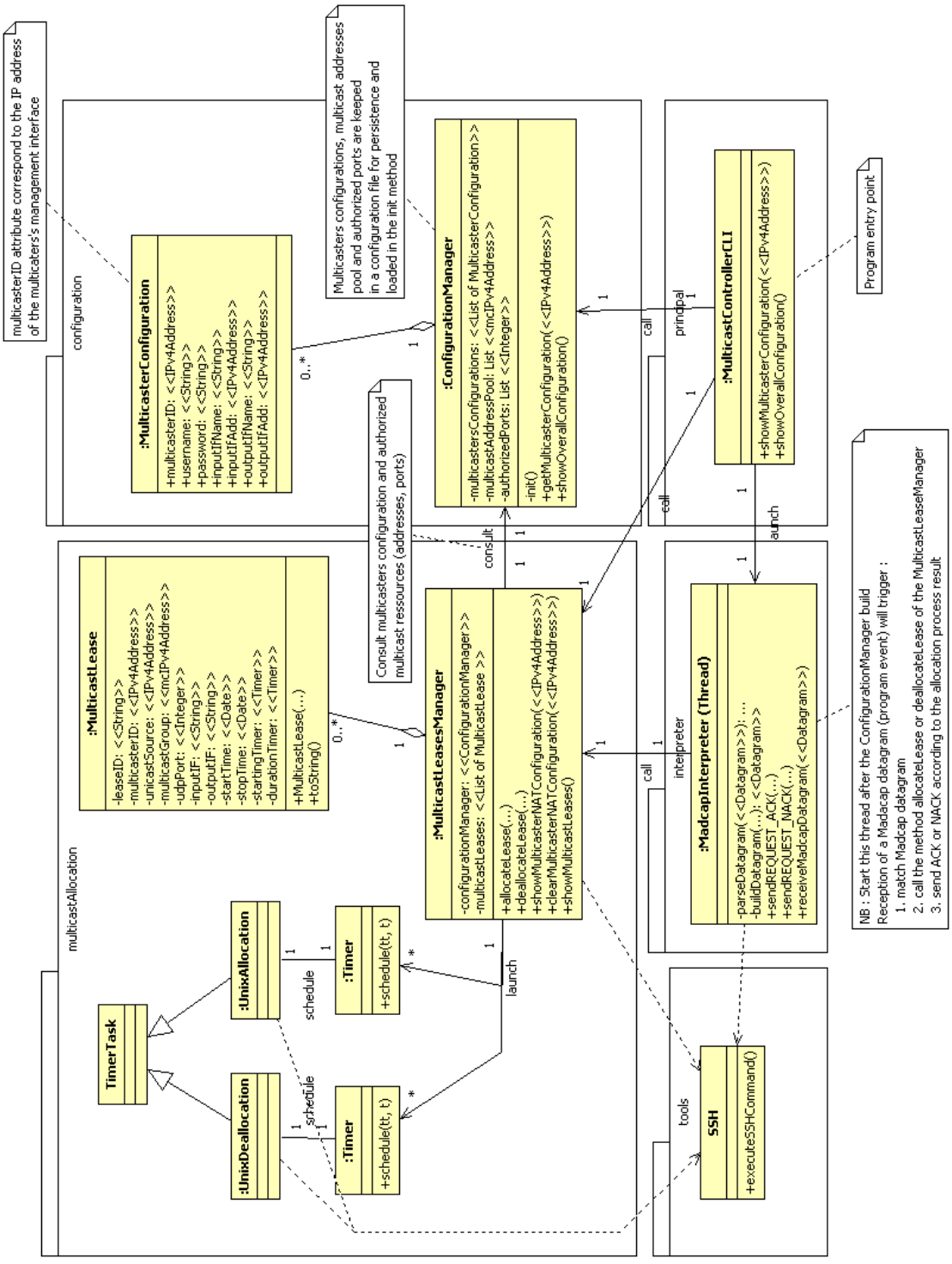


FIG. A.3 – Diagramme de classes du contrôleur multicast

A.2 Configuration d'un multicasteur sous GNU/Linux

Configuration complète d'un « multicasteur » sous GNU/Linux

par Damien LE QUÉRÉ <damien.lequere@orange-ftgroup.com>
encadré par Irène GROSCLAUDE <irene.grosclaude@orange-ftgroup.com>

Stage du 14/03/2011 au 20/08/2011
Orange Labs Lannion, FT/OLNC/RD/CORE/TPN/MOVI

Résumé

Dans le cadre du projet Envision, l'équipe MOVI a besoin de mettre en place un système de diffusion multicast IP à la demande. La solution envisagée pour cela comprend deux types d'éléments : le *multicasteur* et le *contrôleur multicast*. Le *multicasteur* transforme un flux unicast qui lui est destiné en flux multicast dont il sera la nouvelle source vis à vis du domaine multicast. On considérera PIM SM SSM (v2) sur IPv4 pour le routage et IGMPv3 pour la gestion des abonnements. Le *contrôleur multicast*, lui, gère un pool de ressources multicast. Il doit être capable de trouver les ressources multicast disponibles pour le créneau horaire demandé et selon les critères voulus, d'allouer puis de désallouer dynamiquement le multicasteur concerné le moment venu.

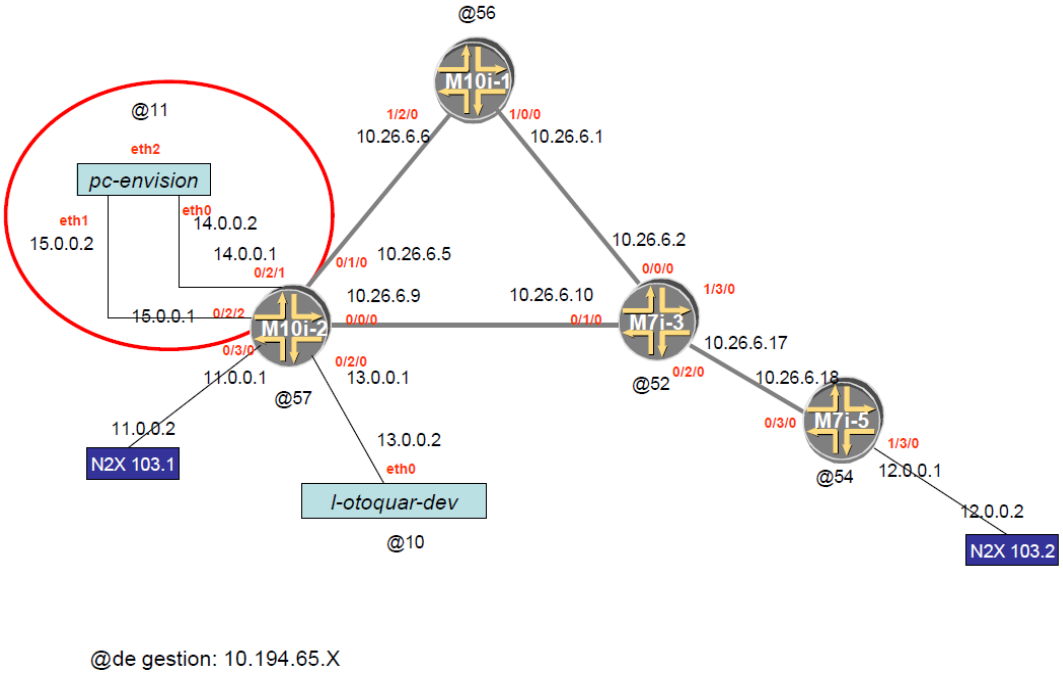
Cette note technique regroupe les procédures permettant la configuration d'un multicasteur avec double translation d'adresses, routage et filtrage des paquets. Tout la configuration étant **statique.**

Objectif

On souhaite employer 'pc-envision' comme multicasteur, et le déployer sur une plate-forme selon la configuration ci-après (voir figure ci-après). On va transformer un flux unicast provenant de la patte de N2X 103.2 et à destination de l'interface eth0 de pc-envision en un flux multicast à destination du groupe 232.0.0.1 et dont la source (PIM SM SSM) sera l'interface eth1 de pc-envision par laquelle il sera émis. La patte 103.2 du N2X se comportant comme un host IGMP (émulation), il s'abonnera au flux (S,G) adéquat.

On considérera la configuration PIM SM SSM de la plate-forme déjà faite, et l'émulation du host IGMPv3 réglée sur le N2X (patte 103.2). On suivra quatre étapes :

1. Configuration du multicasteur avant le déploiement
2. Déploiement sur la plate-forme
3. Configuration du multicasteur après le déploiement
4. Configuration du multicasteur pour la translation d'un flux
5. Tests avec un trafic N2X
6. Références



Déploiement sur la plate-forme MOVI

1- Configuration du multicasteur avant le déploiement

1. Pré-requis :
   - Une distribution **Debian** installée : problèmes de multicast forwarding sous Ubuntu avec *smcroute* mais peut-être pas uniquement voir références plus bas.
     - (Version du Noyau>2.4)
     - une session root
     - Testé avec Debian 6.0.2.1 (version 'stable'), architecture amd64
   - Un accès internet : configurer le proxy pour le gestionnaire de paquets *apt* (voir sur le web)
   - Le dépôt principal de debian dans le sourcelist du gestionnaire *apt* (voir sur le web)
2. Installation des paquets (ici sous debian) :

```
$ sudo su
# apt-get update
# apt-get install aptitude
# apt-get install ssh
# apt-get install smcroute
# apt-get install tshark
```

3. Configuration persistante de l'interface de gestion (après un redémarrage) :
   Dans le fichier /etc/network/interfaces, ajouter :

```
auto eth2
iface eth2 inet static
      address 10.194.65.11
      netmask 255.255.255.192
      gateway 10.194.65.1
      hostname pc-envision
```

4. Nom de la machine (normalement demandé au cours de l'installation de *debian*)

modifier les fichiers */etc/hostname* et */etc/hosts* (ici on choisira « pc-envision »)

5. Pour racourcir le temps d'établissement de la connexion ssh :
   Dans le fichier /etc/ssh/sshd_config, ajouter :

```
UseDNS no
```

2- Déploiement sur la plate-forme (routeurs Juniper)

1. Branchements multicasteur avec le routeur : voir figure 3.1 plus haut
2. Configuration du sous-réseau IP sur le routeur voisin (M10i-2) :

```
configure
set interfaces fe-0/2/1 unit 0 family inet address 14.0.0.1/30
set interfaces fe-0/2/1 unit 0 family inet address 14.0.0.1/30
```

On peut à ce stade utiliser le contrôleur pour configurer le multicasteur de façon rapide et plus simple (voir ci-après « configuration à l'aide du multicast contrôleur »). On configurer manuellement (continuer sur les parties 3 et 4)

3- Configuration du multicasteur après le déploiement

connexion ssh :

```
ssh root@10.194.65.11
```

en mode root :

```
echo 1 > /proc/sys/net/ipv4/ip_forward     # Activation du forwarding IP

ifconfig eth0 14.0.0.2/30                  # Configuration des interfaces
ifconfig eth1 15.0.0.2/30

smcroute -k          # Rédémarrage de smcroute pour ré-initialiser les interfaces  virtuelles
smcroute -d

cat /proc/net/ip_mr_vif      # On peut vérifier que les interface virtuelles on été créées
                             # (elles doivent toutes apparaître)
```

NB : tout redémarrage de la machine oblige à relancer ces commandes avant les commandes de translation du flux. On pourrait rendre la configuration des interfaces persistante en éditant le fichier /etc/network/interfaces comme pour l'interface de gestion. Dans ce cas *smcroute* créerait les interfaces virtuelles correspondantes au démarrage (cependant, moins pratique pour les tests)

4- Configuration du multicasteur pour la translation d'un flux

On considérera la translation suivante :

- de (11.0.0 .2 → 14.0.0.2) vers (15.0.0.2 → 232.0.0.1)
- avec une interface d'entrée eth0 (14.0.0.2) et une interface de sortie eth1 (15.0.0.2)

Commandes linux (mode root)

```
# Translation d'adresses
iptables -t nat -A PREROUTING -i eth0 -s 11.0.0.2 -d 14.0.0.2 -j DNAT --to 232.0.0.1
iptables -t nat -A POSTROUTING -s 11.0.0.2 -d 232.0.0.1 -j SNAT --to-source 15.0.0.2

# Bloquer la transmission par défaut
iptables -P FORWARD DROP

# Autoriser la transmission du flux
iptables -t filter -A FORWARD -s 11.0.0.2 -d 232.0.0.1 -p udp --destination-port 200 -j ACCEPT
```

```
# Ajout de la route multicast statique
smcroute -a eth0 11.0.0.2 232.0.0.1 eth1
```

NB : si la configuration d'une interface utilisée par *smcroute* est modifiée ou si une nouvelle interface est ajoutée, *smcroute* doit être relancé pour en tenir compte, ce qui efface les routes multicast. Il faut donc configurer toutes les interfaces dans un premier temps et ne plus les changer sous peine d'interrompre les trafic multicast courants.

5- Test avec un trafic N2X

Envoi d'un trafic UDP depuis la patte 103.1 dans mon exemple

- IP src = 11.0.0.2
- IP dst = 14.0.0.2
- UDP destination port = 200

Configuration à l'aide du multicast contrôleur :

On peut utiliser le contrôleur pour remplacer la configuration manuelle des parties 3 et 4. Une fois les branchements efféctués, l'interface de gestion du multicasteur configurée de façon permanente, le routeur voisin configuré et la connexion ssh opérationnelle sur une session root, le reste de la configuration du multicasteur peut être gérée avec le contrôleur.

1. Avant tout, pour utiliser le contrôleur, vous devez ajouter le ou les multicasteurs à configurer dans le fichier *mc_settings.conf*.
   (@IP_gestion,
   nom_session_root,
   mot_de_pass,
   nom_IF_entrée,
   @IP_IF_entrée,
   nom_IF_sortie,
   @IP_IF_sortie)
2. Si vous venez de redémarrer, vous constaterez en affichant la configuration du multicasteur que l'ip forwarding n'est pas activé (0) et que les interfaces ne sont pas configurées [menu3]
3. Vous devez donc commencer par l'activer l'ip forwarding et configurer les interfaces
   Ce qui donne avec l'exemple précédent en une seule commande ssh [menu6] :

```
echo 1 > /proc/sys/net/ipv4/ip_forward;ifconfig eth0 14.0.0.2/30 up;ifconfig eth1
15.0.0.2/30 up
```

   nb : ifconfig active les interfaces en même temps qu'elles les configure
4. Les adresses IP nouvellement configurées ne sont pas encore considérées par smcroute (il n'a pas ajouté les interfaces virtuelles) et la queuing discipline n'a pas été créer pour l'interface d'entrée.
   Pour faire tout ça, utiliser le [menu1].
   Vous constaterez ensuite que les interfaces virtuelles ont été ajoutées par smcroute [menu3] et la queuing discipline sur l'interface d'entrée (qdisc)
5. A ce stade, vous être prêt à configurer une nouvelle translation/routage utilisant les interfaces configurées (pour nous eth1 et eth0). Pour cela, utiliser le menu2 qui vous demandera :
   1. @IP du multicasteur (@ de gestion) :10.194.65.11
   2. @IP du serveur unicast : 11.0.0.2
   3. @IP du groupe multicast : 232.0.0.1
   4. Identifiant TC : 1
   5. Numéro port destination : 200
   6. 'date' de début (nombre de secondes avant démarrage) : 0
   7. 'date' de fint (nombre de secondes avant la désallocation) : 3600
   8. Débit maximum (Mbps) : 2
6. Ensuite le programme continu à tourner en attendant une heure avant de désallouer et de se terminer. On peut l'interrompre et abandonner la désallocation automatique pour relancer le programme et

examiner la configuration du multicasteur [menu3]

NB : adresse source-multicast paramétrable

Vous remarquerez que *on ne demande pas l'adresse source du flux multicast*. En effet, dans son état actuel, le programme configurer l'interface de sortie du multicasteur comme la source du flux multicast. Pour que la source soit paramétrable il faut modifier l'allocation linux du contrôleur en conséquence mais aussi ajouter l'état multicast correspondant sur le routeur multicast voisin statiquement (pour qu'il sâche quelle est l'interface de « upstream ») .

6- Références

iptables

- iptables manual : http://linux.die.net/man/8/iptables
- « Linux Firewalls » par Michael Rash, *No Starch Press* 2007

smcroute

- http://www.cschill.de/smcroute/
- http://smcroute.alioth.debian.org/

multicast sous GNU/Linux

- Linux Advanced Routing & Traffic Control HOWTO (chap 8) : http://lartc.org/lartc.html#LARTC.MULTICAST
- IP command manual (ip route, ip mroute, ip maddr) : http://linux.die.net/man/8/ip

pimd

- http://vmlinux.org/jocke/pimd.shtml : http://vmlinux.org/jocke/pimd.shtml
- http://www.linux-networks.net/New.Riders-Linux.Routing/ch06lev1sec1.htm : http://www.linux-networks.net/New.Riders-Linux.Routing/ch06lev1sec1.htm

netfilter extensions

- http://www.netfilter.org/documentation/HOWTO//netfilter-extensions-HOWTO.html
- http://code.google.com/p/iptroute/

problèmes de multicast forwarding sous linux ***(long à débrouiller)***

- http://kerneltrap.org/node/6565
- http://kerneltrap.org/node/8157

A.3 Note technique sur le contrôleur multicast

Note technique sur le multicasteur

par Damien LE QUÉRÉ <damien.lequere@orange-ftgroup.com>
encadré par Irène GROSCLAUDE <irene.grosclaude@orange-ftgroup.com>

Stage du 14/03/2011 au 20/08/2011
Orange Labs Lannion, FT/OLNC/RD/CORE/TPN/MOVI

Résumé

Dans le cadre du projet Envision, l'équipe MOVI a besoin de mettre en place un système de diffusion multicast IP à la demande. La solution envisagée pour cela comprend deux types d'éléments : le *multicasteur* et le *contrôleur multicast*. Le *multicasteur* transforme un flux unicast qui lui est destiné en flux multicast dont il sera la nouvelle source vis à vis du domaine multicast. On considérera PIM SM SSM (v2) sur IPv4 pour le routage et IGMPv3 pour la gestion des abonnements. Le *contrôleur multicast*, lui, gère un pool de ressources multicast. Il doit être capable de trouver les ressources multicast disponibles pour le créneau horaire demandé et selon les critères voulus, d'allouer puis de désallouer dynamiquement le multicasteur concerné le moment venu.

Cette note technique regroupe les procédures permettant la configuration d'un multicasteur avec double translation d'adresses, routage et filtrage des paquets. Tout la configuration étant **statique.**

Première implémentation (Python)

- Version de démonstration
- Sans interpréteur Madacap
- Interface en ligne de commande pour déclencher l'allocation

Outils

- Python 2.6.6
- Module ssh : python-paramiko (v1.7.6-2)
- Module adressage IP : python-ipaddr (v2.1.4)

Références

Python :

- Documentation python en ligne : http://docs.python.org/release/3.1.3/
- IP address manipulation library : http://www.python.org/dev/peps/pep-3144/
  - Wiki : http://code.google.com/p/ipaddr-py/wiki/UpgradingTo2x
  - Documentation en ligne : http://packages.python.org/ipaddr/

Paramiko (ssh sur python) :

- Site officiel : http://www.lag.net/paramiko/
- Tutorial en ligne : http://jessenoller.com/2009/02/05/ssh-programming-with-paramiko-completely-different/
- Documentation en ligne : http://www.lag.net/paramiko/docs/

Implémentation

- Le script : « MulticastControllerCLI.py »
- Le fichier de configuration : « mc_settings.conf »

Deuxième implémentation (Java)

Outils

- Java : *java-1.6.0-openjdk-1.6.0*
- Gestion de l'addressage IP : *java.net.InetAddress* de openjdk
- Connexion SSH : *sshtools* (v0.2.9) (dernière mise à jour le 27/04/2011)
- Gestion des messages logs : *apache-log4j*

Références

- *sshtools* : Site officiel : http://sourceforge.net/projects/sshtools/
- *apache-log4j* : Site officiel : http://logging.apache.org/log4j/1.2/

État d'avancement du contrôleur

Travail fait :

Le contrôleur multicast est encore en développement. Il a vocation à être intégré à une brique applicative plus importante de *Envision*. Dans son état actuel il est utilisable pour des tests avec une interface textuelle (menu).

```
1- Clear a multicaster configuration
2- Reserve a multicast lease
3- Show a multicaster configuration
4- Show the multicast controller configuration
5- Show curents multicast leases
6- Execute an Unix command on a multicaster
7- Exit
```

Ce qui correspond actuellement à :

1. Réinitialiser un multicasteur
2. Déclencher la translation d'un flux selon :
   - la source du flux unicast
   - le port de destination
   - le protocole de transport (udp)
   - les dates de début et de fin (actuellement en secondes pour les tests)

   Le multicasteur et le groupe multicast sont pour l'instant choisi arbitrairement
3. Voir la configuration d'un multicasteur en affichant :
   - Les translations d'adresses (affichage *iptables*)
   - Les règles de firewall (affichage *iptables*)
   - Le routage multicast statique (*ip mroute show*)
   - Le contrôle de trafic (affichage des classes, filtres etc de *tc*)
4. Voir la configuration du multicast contrôleur en affichant :

- Les multicasteurs enregistrés
- Les groupes multicast autorisés
- Les ports autorisés

5. Voir les réservation multicast courantes
   N'affiche rien actuellement car le programme permet de lancer une seule option du menu avant de quitter.
6. Exécuter une commande linux en mode root sur un multicasteur (pratique pour les tests)

Travail à faire :

- Poursuivre le développement de l'interpréteur de requête du protocole CINA
- Gérer l'allocation des ressources dans le temps (créneaux horaires)
- Choisir les ressources intelligemment : multicasteur, groupe multicast, source
- Développer un module pour récupérer les informations de monitoring pour l'allocation intelligente (pas besoin si le module de monitoring est intégré au même ensemble applicatif)
- Gérer la connexion SSH aux multicasteurs avec des clés plutôt que des login/mot de pass (modifier le code du contrôleur en conséquence)

Aperçu du menu :

